

ELEC520

Python programming

Summer 2026

파이썬 소개

<https://colab.research.google.com/drive/1HFCdcwC1AydKO2pOSJv34qd1OWSoUsPq?usp=sharing>

프로그램

- 컴퓨터가 실행할 명령어들을 조직적으로 모아 놓은 것
 - 컴퓨터는 0과 1의 이진 값 만을 저장하고 이해
 - 컴퓨터가 수행하는 명령은 001110010001000... 의 값으로 되어 있음

- **프로그래밍**

- 하나 이상의 명령어들을 입력하여 프로그램을 작성하는 과정
- 다른 표현으로 '코딩'

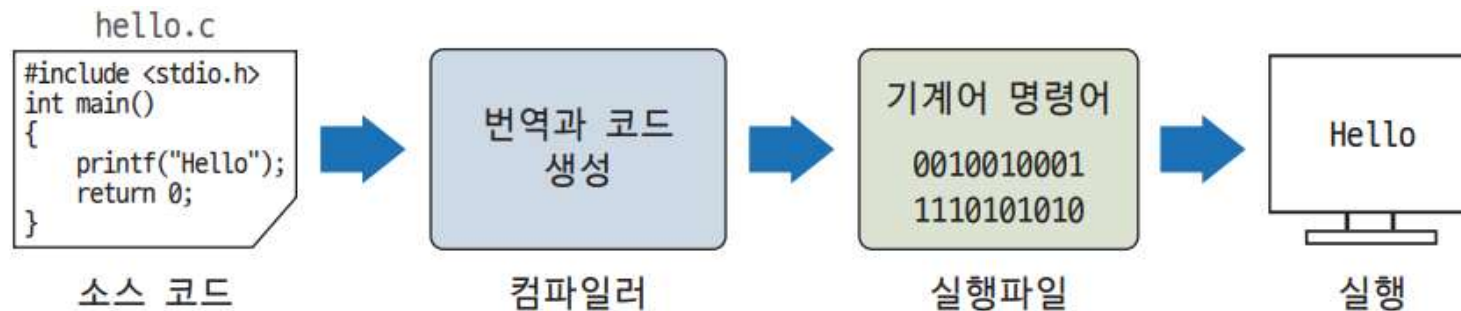
English:	ADD	contents of ADDRESS1	and contents of ADDRESS2	then store result in ADDRESS3
Machine language:	01011010	00000001	00000010	00000011
Assembly language:	add	a1,	a2,	a3
High-level language:	total = number1 + number2			

프로그래밍 언어

- 01001000100...과 같은 명령은 사람이 이해하기 어렵고 작성 시간이 오래 걸리며 오류가 많고 수정이 힘들
- 사람이 이해할 수 있는 프로그래밍 언어를 이용하여 명령 입력
 - 컴파일러 혹은 인터프리터가 필요함
- 자동차나 프린터 등의 미리 틀이 정해진 기계(임베디드 시스템 Embedded System)에서 제한된 목적과 용도로 사용되는 프로그래밍 언어는 수행 속도가 중요하므로 C 언어를 주로 사용
- 파이썬은 강력하면서도 배우기 쉬움

컴파일 방식

- 프로그램 명령어를 기계어로 번역한 후 이 기계어를 실행하는 방식
- C, C++, JAVA 언어 등이 있음



[그림 1-8] 컴파일 방식의 프로그램이 동작하는 원리: 컴파일러는 소스 코드를 번역하여 기계어 명령어를 생성한다. 이 기계어는 컴퓨터에서 실행되어 결과를 만든다.

인터프리터 방식

- 프로그램 명령어를 한 번에 한 라인 씩 읽어 번역한 후 바로 실행
- 파이썬, BASIC 등의 언어가 있음



[그림 1-9] 인터프리터 방식의 프로그램이 동작하는 원리: 인터프리터는 소스 코드를 한 줄씩 읽어들이어서 번역한 후 이 코드를 실행하는 방식으로 동작한다.

인터프리터 방식과 컴파일 방식의 비교

	인터프리터 방식	컴파일 방식
정의	명령어들을 한 번에 한 줄씩 읽어 실행	명령어를 기계어로 번역하는 방식
장점	컴파일 단계를 거칠 필요가 없음	일반적인 경우 속도가 더 빠름
단점	실행 시간이 느림	코드가 길다면 컴파일에 상당한 시간이 소요
사용되는 언어	파이썬, BASIC 등	C, JAVA, FORTRAN, PASCAL 등

파이썬

- 1989년 귀도 반 로섬(Guido Van Rossum)에 의해 개발
- 인터프리터 방식의 객체지향 프로그래밍 언어



[그림 1-5] 파이썬을 개발한 귀도 반 로섬
(출처: 위키백과)



[그림 1-6] Python 로고

소스 코드

- 소스 코드 source code
 - 프로그래밍 언어로 작성된 명령어들의 목록
- 소스 파일 source file
 - 소스 코드가 저장된 파일
 - 스크립트: 파이썬 소스 파일

< 소스코드 예시 >

```
age = input('나이를 입력하세요')  
  
if int(age) < 19:  
    print('할인되었습니다.')  
else:  
    print('할인이 안 됩니다.')
```

정해진 문법에 맞게 명령을 입력하면
컴퓨터는 이 코드를 실행하여 결과를 보여준다

파이썬의 특징

- 편리한 built-in functions.
- 오픈 소스로 방대한 라이브러리를 무료로 편리하게 이용 가능
- 높은 생산성: 짧은 코드로 많은 기능 수행 가능
- 직관적이고 단순한 문법
- 들여쓰기를 통한 블록 구분: {} 사용 x, 세미콜론 x
- 객체 지향
- 동적 타이핑
- ...

Python 3.12.2 (tags/v3.12.2:6abddd9, Feb 6 2024, 21:26:36) [MSC v.1937 64 bit (AMD64)] on win32

Type "help", "copyright", "credits" or "license" for more information.

>>> import this

The Zen of Python, by Tim Peters

Beautiful is better than ugly.

Explicit is better than implicit.

Simple is better than complex.

Complex is better than complicated.

Flat is better than nested.

Sparse is better than dense.

Readability counts.

Special cases aren't special enough to break the rules.

Although practicality beats purity.

Errors should never pass silently.

Unless explicitly silenced.

In the face of ambiguity, refuse the temptation to guess.

There should be one-- and preferably only one --obvious way to do it.

Although that way may not be obvious at first unless you're Dutch.

Now is better than never.

Although never is often better than *right* now.

If the implementation is hard to explain, it's a bad idea.

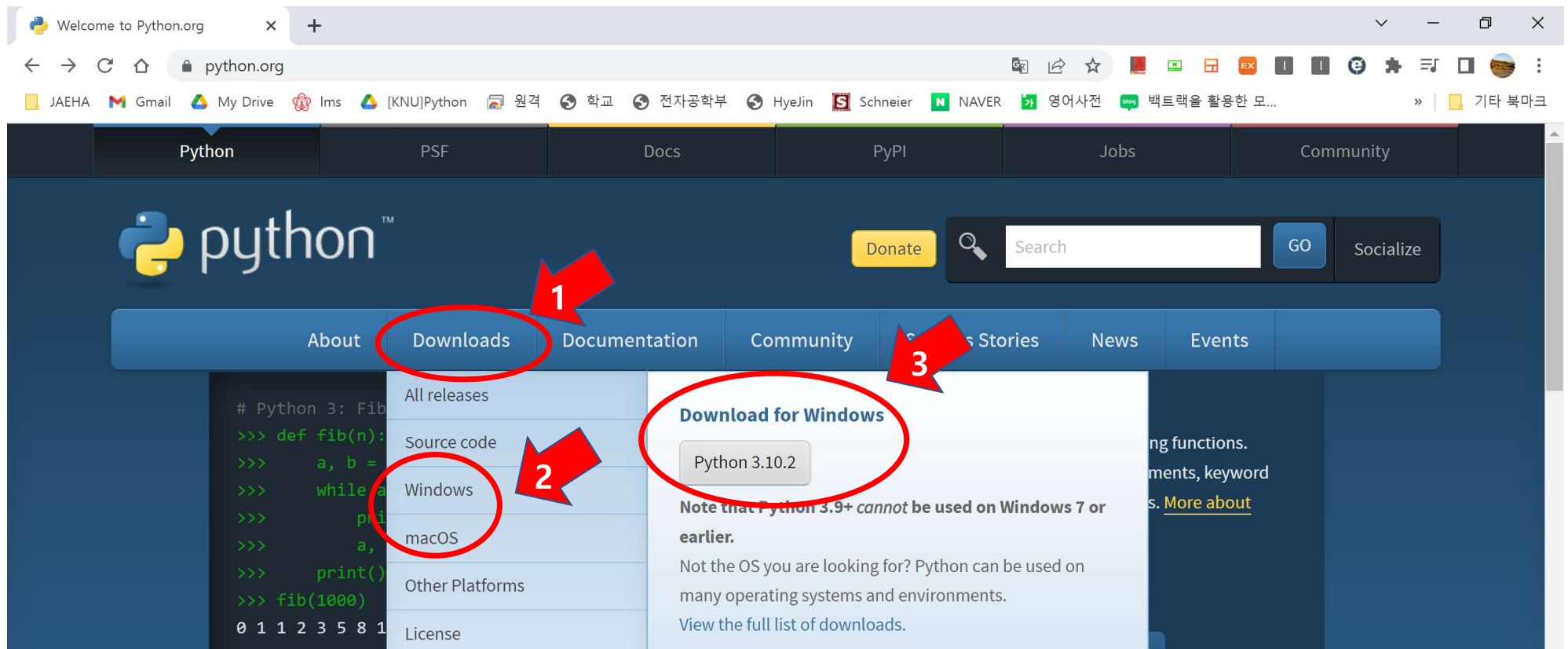
If the implementation is easy to explain, it may be a good idea.

Namespaces are one honking great idea -- let's do more of those!

>>> |

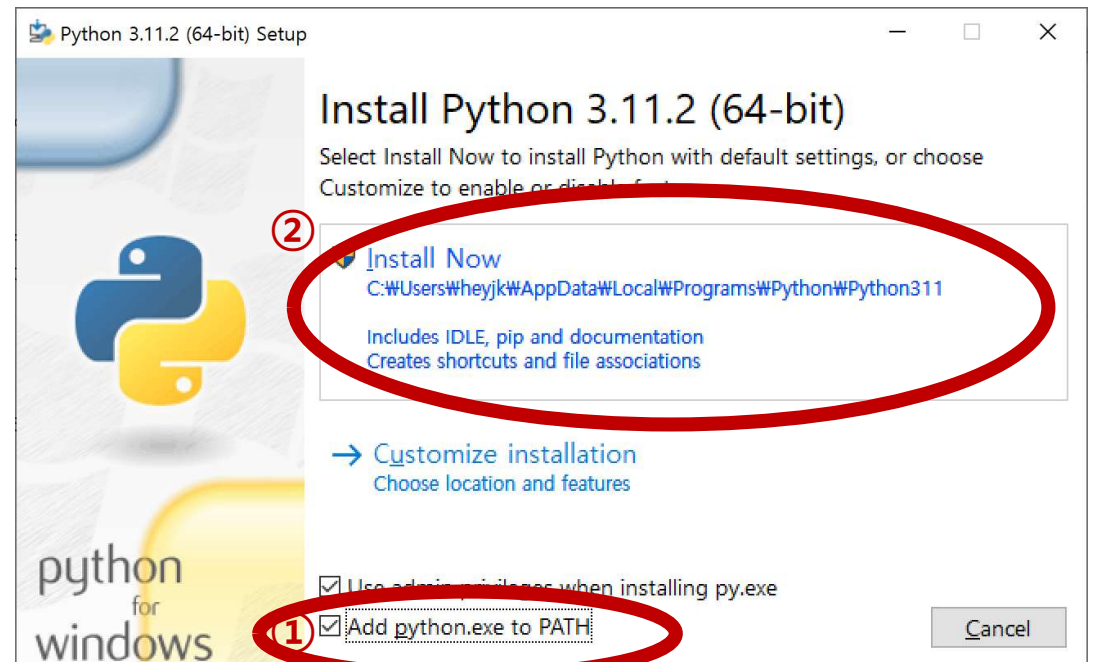
파이썬 다운로드

- python.org



파이썬 설치(windows 기준)

- 다운 받은 설치 파일 실행
- 체크박스 선택
 - Install launcher for all users(recommended)
 - Add Python to PATH
- Install Now 선택



파이썬 실행

• 대화형 모드

- 대화창에서 즉각적인 피드백을 받을 수 있는 모드
- 간단한 코드 테스트용
- 파이썬 대화창, 운영체제 대화창(콘솔창, cmd), IDLE 통해 대화형 모드 가능

• 파일 모드(스크립트 모드)

- .py 확장자를 가지는 스크립트를 만들어 명령어 저장
- 복잡한 로직이 있는 코드는 스크립트 파일에 저장 후 실행
- 운영체제 대화창(cmd), IDLE 통해 대화형 모드 가능

파이썬 대화창

- 프롬프트에 커서가 깜빡인다면 입력 받을 준비가 되었다는 것
- **프롬프트 prompt**: 사용자의 입력을 받을 준비가 되면 화면에 나타나는 신호
- 명령을 입력하면 인터프리터가 해석 후 실행

파이썬에서는
>>>이
프롬프트

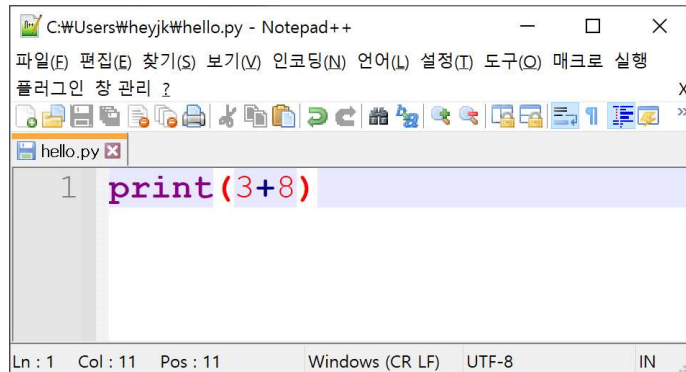
파이썬 대화창

```
Python 3.10 (64-bit)
Python 3.10.1 (tags/v3.10.1:2cd268a, Dec 6 2021, 19:10:37) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> 3+2
5
>>> print('hello, python!')
hello, python!
>>> -
```

커서

파일 모드(feat. 콘솔창)

- 대화형 모드는
 - 명령어 테스트 및 배우기 용이
 - 명령 입력이 편리하나 명령문이 저장되지 않고 재사용이 불가능
- 명령어를 텍스트 파일에 따로 저장시켜 이 파일을 실행
- 파일에 코드 작성 후 콘솔창에서 실행



C:\Users\heyjk\hello.py - Notepad++

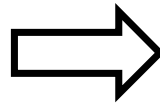
파일(E) 편집(E) 찾기(S) 보기(V) 인코딩(N) 언어(L) 설정(I) 도구(Q) 매크로 실행

플러그인 창 관리 ?

hello.py

```
1 print(3+8)
```

Ln : 1 Col : 11 Pos : 11 Windows (CR LF) UTF-8 IN



선택 C:\Windows\System32\cmd.exe

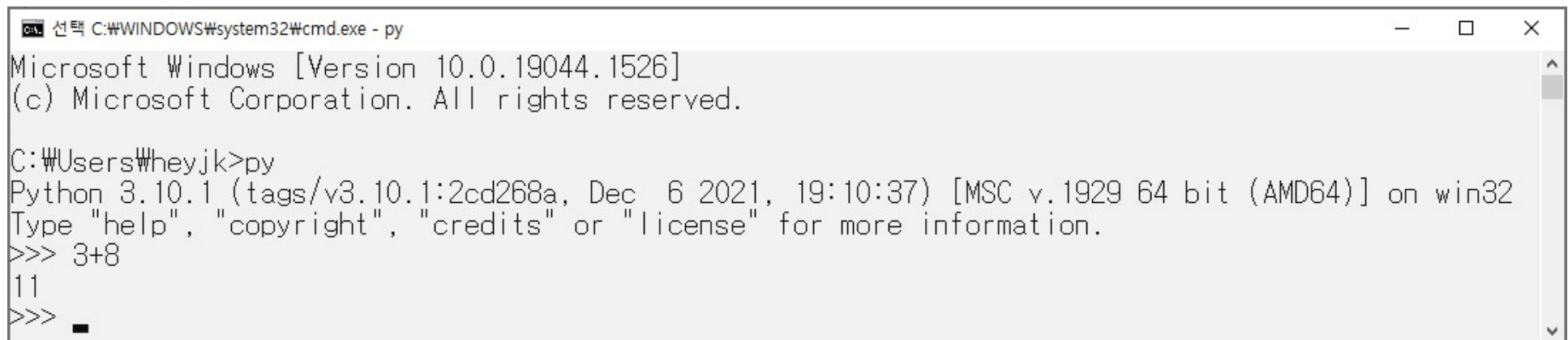
Microsoft Windows [Version 10.0.19044.2604]
(c) Microsoft Corporation. All rights reserved.

C:\Users\heyjk>py hello.py

11

콘솔에서 대화형 모드

- 커맨드라인에 python 혹은 py 입력하면 대화형 모드 진입



```
C:\> 선택 C:\WINDOWS\system32\cmd.exe - py
Microsoft Windows [Version 10.0.19044.1526]
(c) Microsoft Corporation. All rights reserved.

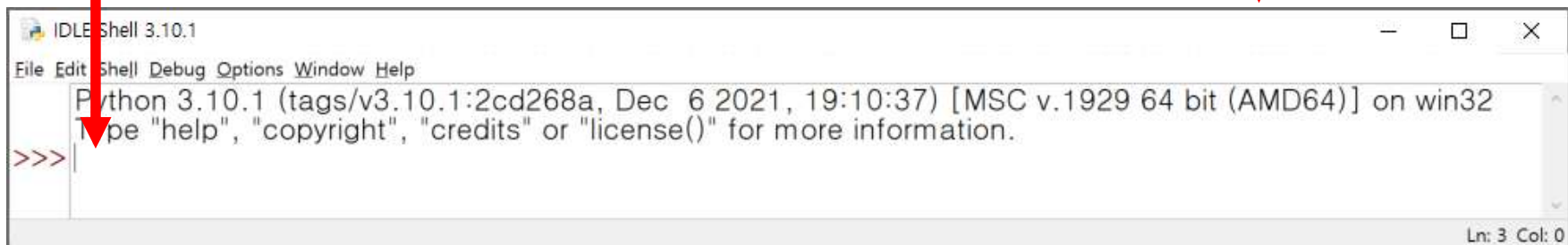
C:\Users\heyjk>py
Python 3.10.1 (tags/v3.10.1:2cd268a, Dec  6 2021, 19:10:37) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> 3+8
11
>>> _
```


IDLE

- 파이썬에서 기본으로 제공하는 통합 개발 환경(IDE)
- **I**ntegrated **D**evelopment and **L**earning **E**nvironment
- 간단한 명령어 및 함수 테스트 시엔 대화 모드로 사용

프롬프트에 명령어 입력
하여 대화형 모드 가능

파이썬 셸 `python shell`



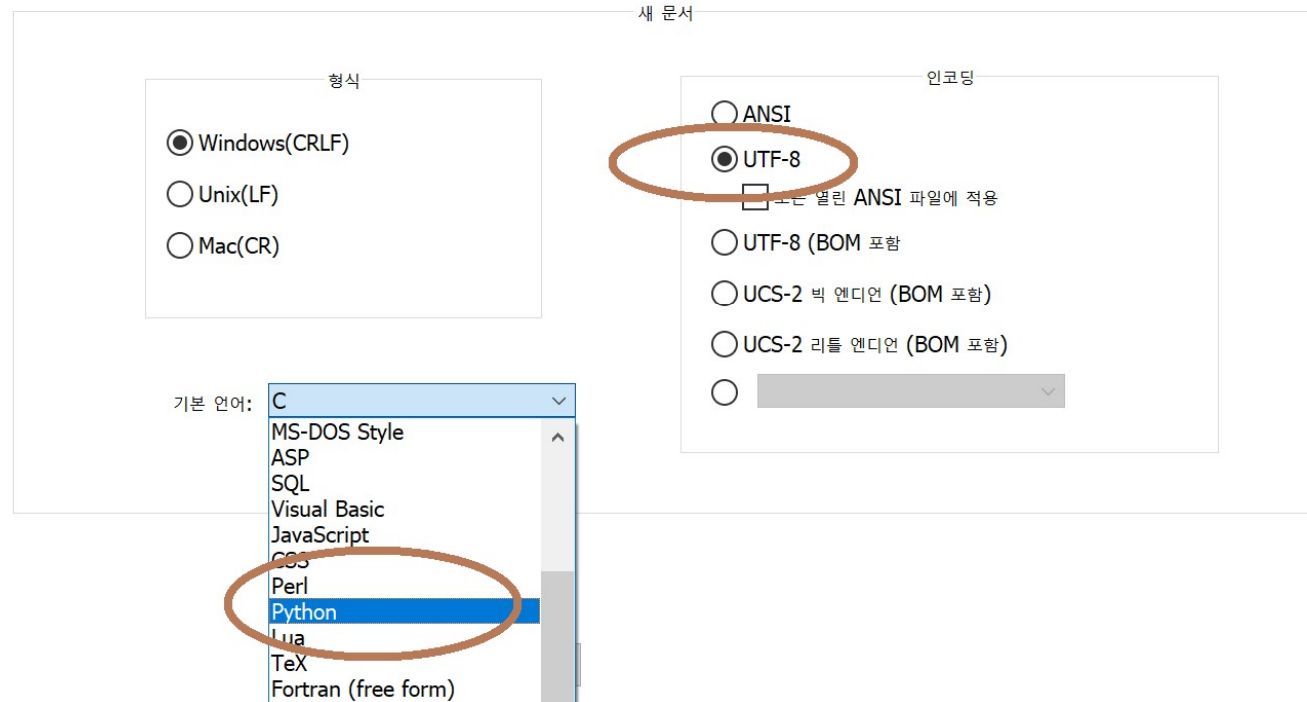
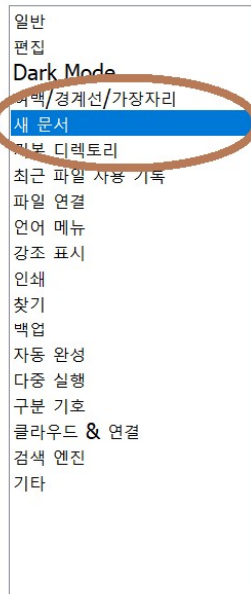
The screenshot shows the IDLE Shell 3.10.1 window. The title bar reads 'IDLE Shell 3.10.1'. The menu bar includes 'File', 'Edit', 'Shell', 'Debug', 'Options', 'Window', and 'Help'. The main text area displays the Python version and build information: 'Python 3.10.1 (tags/v3.10.1:2cd268a, Dec 6 2021, 19:10:37) [MSC v.1929 64 bit (AMD64)] on win32'. Below this, it says 'Type "help", "copyright", "credits" or "license()" for more information.' The prompt '>>>' is visible on the next line. A red arrow points from the text box on the left to the prompt. Another red arrow points from the text box on the right to the 'python shell' text in the title bar.

```
IDLE Shell 3.10.1
File Edit Shell Debug Options Window Help
Python 3.10.1 (tags/v3.10.1:2cd268a, Dec 6 2021, 19:10:37) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
```

Ln: 3 Col: 0

Notepad++ 환경 설정

기본 설정



출력 함수와 변수

print()

input()

변수와 식별자

print() 함수: Hello, World! 출력

- print()함수의 인자는 문자열(작은 따옴표, 혹은 큰 따옴표로 묶음)이나 수식

```
>>> print('Hello World!')
```

문장, 표현문, 표현식

```
Hello World!
```

```
>>> print('Hello', 'World!')
```

```
Hello World!
```

쉼표로 구분되어 있을 경우 공백을 삽입하고 다음 문자열을 출력

```
>>> print('Hello'+'World!')
```

```
HelloWorld!
```

덧셈기호로 문자열을 연결할 경우 공백이 삽입되지 않음

```
>>> print(100)
```

```
100
```

```
>>> print(2+4)
```

```
6
```

2 + 4 연산의 결과가 출력됨

print() 함수(cont.)

```
>>> print(Hello Python!!)
```

```
...
```

```
SyntaxError: invalid syntax
```

오류 발생

SyntaxError : 구문오류

invalid syntax : 유효하지 않은 구문

번역기는 여러분에게 친절하게 오류를 표시해 주고 알려줍니다

```
>>> print('My age is', 20)
```

```
My age is 20
```

```
>>> print('오늘의 걸음 수', 8000, '걸음')
```

```
오늘의 걸음 수 8000 걸음
```

제대로 된 출력방식

문자열과 숫자는 쉼표로 구분해 줍시다

```
>>> print('Hello ' * 2)
```

```
Hello Hello
```

```
>>> print('Hello ' * 4)
```

```
Hello Hello Hello Hello
```

문자열에 * 연산을 하고 숫자를 넣어 줄 경우 :
숫자만큼 문자열을 반복 출력한다

원의 반지름, 면적, 둘레 출력하기

circle.py

```
print('원의 반지름', 4.0)  
print('원의 면적', 3.14 * 4.0 * 4.0)  
print('원의 둘레', 2.0 * 3.14 * 4.0 )
```

실행결과

원의 반지름 4.0
원의 면적 50.24
원의 둘레 25.12

반지름 변경

- 방금 작성한 프로그램에서 반지름이 5.0, 6.0 인 원의 면적과 둘레를 새로 구하는 경우 다음과 같이 코드를 수정해야 함

```
print('원의 반지름', 5.0)
print('원의 면적', 3.14 * 5.0 * 5.0)
print('원의 둘레', 2.0 * 3.14 * 5.0)
```

```
print('원의 반지름', 6.0)
print('원의 면적', 3.14 * 6.0 * 6.0)
print('원의 둘레', 2.0 * 3.14 * 5.0)
```



번거로운 작업이다
오류의 가능성이 크다

논리 오류 : 문법은 맞지만 의도한 바와 다른 결과
의도한 바는 원의 반지름이 6.0일때의 둘레임

변수 **variable**

- 프로그램을 작성하다 보면 계속해서 값을 변경시켜 주어야하는 경우 하나라도 실수를 하게 되면 프로그램의 오류가 나타남
- 변수를 도입해서 번거로운 일을 간단하게 만들 수 있음
 - radius = 4.0 와 같이 radius 라는 이름을 가지는 변수 생성
 - radius는 4.0이라는 값을 참조(가리킴)
- **변수**
 - 데이터가 저장된 메모리 공간
 - 할당(대입)문을 사용하여 변수 선언
 - 변수의 이름을 통해 데이터에 접근

변수 사용하여 원의 정보 출력

circle_with_var.py

```
radius = 4.0
print('원의 반지름', radius)
print('원의 면적', 3.14 * radius * radius)
print('원의 둘레', 2.0 * 3.14 * radius)
```

실행결과

원의 반지름 4.0
원의 면적 50.24
원의 둘레 25.12

circle_with_var.py (수정)

```
radius = 6.0
print('원의 반지름', radius)
print('원의 면적', 3.14 * radius * radius)
print('원의 둘레', 2.0 * 3.14 * radius)
```

변수에 값을 저장하고 이를 불러서 사용하면
프로그램의 수정이 쉬워지고 오류를 줄일 수 있다

실행결과

원의 반지름 6.0
원의 면적 113.03999999999999
원의 둘레 37.68

식별자

- 변수나 함수, 클래스 등의 이름
- 하나의 변수 이름을 여러 개의 메모리 위치를 지칭하는데 사용하게 되면 어느 메모리 공간을 지칭하는지 알기 어려움
- 따라서 다른 메모리 위치에는 서로 다른 이름을 부여해야 함
- 프로그램이 복잡해지면 walk_distance, num_of_hits, english_dict, student_name과 같이 **그 의미를 명확하게 이해할 수 있는 식별자를 사용**하는 것이 편리하다.

식별자 규칙

- 문자와 숫자, 밑줄 문자 `_`로 구성
- 중간에 공백 불가
- 첫 글자는 숫자 불가
- 대문자와 소문자는 구분된다. 따라서 `Count`와 `count`는 서로 다른 식별자이다.
- 식별자의 길이에 제한은 없다.
- 키워드는 식별자로 사용할 수 없다.

사용 가능한 식별자	특징	사용 불가능한 식별자
<code>number4</code>	영문자로 시작하고 난 뒤에는 숫자를 사용할 수 있음	<code>1st_variable</code>
<code>__code__</code>	밑줄 문자는 일반 문자와 같이 식별자 어디든 나타날 수 있음	<code>my list</code>
<code>my_list</code>		<code>global</code>
<code>for_loop</code>	키워드라 할지라도 다른 문자와 연결해 쓰면 문제가 없음	<code>ver2.9</code>
숫자변수	유니코드 문자인 한글 문자도 변수로 사용 가능	<code>num&co</code>

키워드(예약어)

- 이미 예약된 문자로 미리 지정된 역할을 수행하는 단어
- import, for, if, def, class... 등과 같은 단어가 이에 해당

[표 2-4] 파이썬 키워드 목록: 파이썬 키워드는 사용 용도가 정해져 있어서 변수로 사용할 수 없다.

파이썬의 키워드				
False	class	finally	is	return
None	continue	for	lambda	try
True	def	from	nonlocal	while
and	del	global	not	with
as	elif	if	or	yield
assert	else	import	pass	
break	except	in	raise	

변수의 선언과 할당(대입) 연산자

코드 2-6 : 변수 선언 및 값 변경

change_var.py

name = '홍길동' ← 변수 name 선언, name은 문자열 '홍길동'을 참조.

age = 27 ← 변수 age 선언, age는 정수 27을 참조.

print('안녕! 나는', name, '이야. 나는 나이가', age, '살이야.')

name = '홍길순' ← 이제 name은 문자열 '홍길순'을 참조.

age = 23 ← 이제 age는 정수 23을 참조.

print('안녕! 나는', name, '이야. 나는 나이가', age, '살이야.')

실행결과

안녕! 나는 홍길동 이야. 나는 나이가 27 살이야.

안녕! 나는 홍길순 이야. 나는 나이가 23 살이야.

- 변수 이름 = 상수(리터럴, literal), 변수, 수식

```
>>> 27 = age
```

File "<stdin>", line 1

SyntaxError: can't assign to literal

- 할당 연산자가 처음 나타나는 경우 변수가 선언^{declare}되었다고 함.

대화창 실습 : 다양한 자료형의 이해와 type() 함수

```
>>> num = 85
>>> type(num)
<class 'int'>
>>> pi = 3.14159
>>> type(pi)
<class 'float'>
>>> message = "Good morning"
>>> type(message)
<class 'str'>
```

type() 함수와 동적 타이핑

대화창 실습 : 동적 형결정(타이핑)의 이해

```
>>> foo = 100
>>> type(foo)
<class 'int'>
>>> foo = 'Hello'
>>> type(foo)
<class 'str'>
```

변수 foo는 정수 형 객체를 참조하다가
나중에 문자열 형 객체를 참조할 수 있다
이 변수가 참조하는 자료형은 변경가능하다
이를 동적 형 결정(타이핑)방식이라 한다

주석

- 인터프리터가 해석을 하지 않고 건너뛰는 문장
- 프로그램에 대한 설명
- 한 줄 주석: #
- 여러 줄 주석: `'''`, `"""`(작은/큰 따옴표 3개)

입력함수 input()

실행결과

Enter your name :

Hong GilDong

Hello Hong GilDong !

- input() 함수
 - 사용자가 입력한 값을 **문자열** 형태로 반환하는 함수

코드 4-31 : input() 입력함수를 사용한 name의 입력 방법

input_name1.py

```
print('Enter your name : ')\nname = input()    # 사용자의 키보드 입력 값을 name이 참조함\nprint('Hello', name, '!')
```


입력함수 예제

코드 4-32 : 문자열을 이용한 input 입력함수의 사용법

input_name2.py

```
name = input('Enter your name : ')\nprint('Hello', name, '!')
```

실행결과

```
Enter your name : Hong GilDong\nHello Hong GilDong !
```

- 두 수를 입력 받아 두 수의 합을 계산하는 함수

덧셈과 casting(형변환) 함수

```
>>> 100 + 1    # 정수의 덧셈으로 수+수
101
>>> '100' + '1' # 문자열의 덧셈으로 문자열+문자열 연산을 통해 문자를 연결
'1001'
>>> '100' + 1   # 문자열과 정수의 덧셈(오류 발생)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: must be str, not int
```

대화창 실습 : int() 함수와 float(), str() 함수를 이용한 문자열의 변환

```
>>> int('100') + 1    # 문자열 '100'의 덧셈을 위해 정수(int)로 변환
101
>>> float('100') + 1  # 문자열 '100'을 덧셈을 위해 실수(float)로 변환
101.0
>>> '100' + str(1)    # 문자열 덧셈을 위해서 1을 문자열 '1'로 변환
'1001'
```

실습

<https://colab.research.google.com/drive/1HFCdcwC1AydKO2pOSJv34qd1OWSoUsPq?usp=sharing>

1. 사용자로부터 이름과 나이를 입력 받아 이름과 10년 후 나이를 출력하는 프로그램을 작성하시오.

실행 결과

```
이름: 홍길동
나이: 22
홍길동님의 10년 후 나이는 32세입니다.
```

2. 사용자로부터 직사각형의 가로, 세로 길이를 입력 받아 직사각형의 둘레의 길이와 넓이를 출력하는 프로그램을 작성하시오.

실행 결과

```
직사각형의 가로: 4
직사각형의 세로: 3
둘레의 길이: 14
넓이: 12
```